

Μεταγλωττιστές 2025-26

Προσθήκη συναρτήσεων στη γραμματική
των αριθμητικών εκφράσεων

Στόχος

- Η υποστήριξη συναρτήσεων από τον συντακτικό αναλυτή
 - Και AST interpreter/TAC generator
- Δηλώνονται στην **αρχή** του κώδικα

```
function func-name ( argument-list ) block-statement
```
- Το κυρίως πρόγραμμα ακολουθεί **μετά τις δηλώσεις των συναρτήσεων**
- Τόσο οι δηλώσεις των συναρτήσεων όσο και το κυρίως πρόγραμμα **μπορεί να είναι κενά**
 - Ένα έγκυρο πρόγραμμα στη γλώσσα μας μπορεί να μην περιέχει τίποτα

Στόχος

- Κλήση συναρτήσεων

func-name (expression-list)

- Καλούνται ως **statements** (εντολές)
 - Τυχόν επιστρεφόμενη τιμή αγνοείται
- Καλούνται ως **expressions** (ως μέρος μιας έκφρασης)
 - Πρέπει να επιστρέφουν μια float τιμή

- Επιστροφή από συνάρτηση

- Στο φυσικό τέλος μιας συνάρτησης
- Εντολή return

return expression

- Για λόγους απλότητας της γραμματικής, το return επιτρέπεται ακόμα και στο κυρίως πρόγραμμα
- Και το expression μετά το return είναι υποχρεωτικό

Προσθήκες στη γραμματική (1)

```
Program → F_declarations Stmt_list  
F_declarations → F_declaration F_declarations | ε  
F_declaration → function id ( Param_list ) Block_stmt  
Param_list → id Param_tail | ε  
Param_tail → , id Param_tail | ε
```

- **1^ο μέρος:** προσθήκη πριν το Stmt_list
 - **F_declarations:** Προηγούνται οι δηλώσεις των συναρτήσεων (0 ή περισσότερες)
 - **Stmt_list:** Το κυρίως πρόγραμμα που ακολουθεί (μπορεί επίσης να είναι κενό)
 - **Param_list:** μηδέν ή περισσότερες **παράμετροι** στον ορισμό μιας συνάρτησης

Προσθήκες στη γραμματική (2)

```
Stmt → id Assign_or_call | print Lexpr  
      | if Lexpr Block_stmt (else Block_stmt)?  
      | while Lexpr Block_stmt  
      | return Lexpr  
Assign_or_call → = Lexpr | ( Arg_list )
```

- 2^ο μέρος: προσθήκες στο Stmt
 - Προσθήκη κλήσης συνάρτησης ως statement
 - **Arg_list**: μηδέν ή περισσότερα **ορίσματα** (εκφράσεις Lexpr) στην κλήση μιας συνάρτησης
 - Προσθήκη εντολής (statement) return

Προσθήκες στη γραμματική (3)

Factor $\rightarrow$ (Lexpr) | id Id_or_call | number

Id_or_call $\rightarrow$ (Arg_list) | ϵ

Arg_list $\rightarrow$ Lexpr Arg_tail | ϵ

Arg_tail $\rightarrow$, Lexpr Arg_tail | ϵ

- 3^ο μέρος: προσθήκες στο Factor
 - Για την υποστήριξη της κλήσης συναρτήσεων ως μέρη έκφρασης (**expression**)
 - **Arg_list**: μηδέν ή περισσότερα **ορίσματα** (εκφράσεις Lexpr) στην κλήση μιας συνάρτησης

Μη τερματικό	Σύνολο FIRST	Σύνολο FOLLOW
Program	function id print if while return	#
F_declarations	function	id print if while return #
F_declaration	function	
Param_list	id	)
Param_tail	,	)
Stmt_list	id print if while return	} #
Stmt	id print if while return	
Assign_or_call	= (	
Block_stmt	{ id print if while return	
Lexpr	not (id number	
Lterm_tail	or	) { } , function id print if else while return #
Lterm	not (id number	

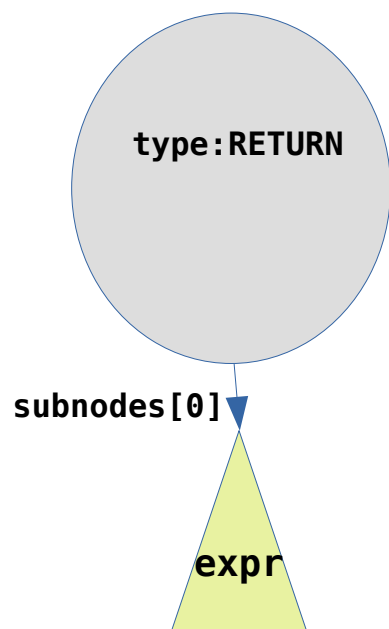
Μη τερματικό	Σύνολο FIRST	Σύνολο FOLLOW
Lfactor_tail	and	) { } , function or id print if else while return #
Lfactor	not (id number	
Rest	== != > >= < <=	) { } , function and or id print if else while return #
Expr	(id number	
Term	(id number	
Factor	(id number	
Id_or_call	(	* / + - == != > >= < <= and or { } ,) function id print if while return else #
Arg_list	not (id num	)
Arg_tail	,	)
Addop	+ -	
Multop	* /	
Relop	== != > >= < <=	

Νέα tokens: , function return

Δοκιμαστική είσοδος

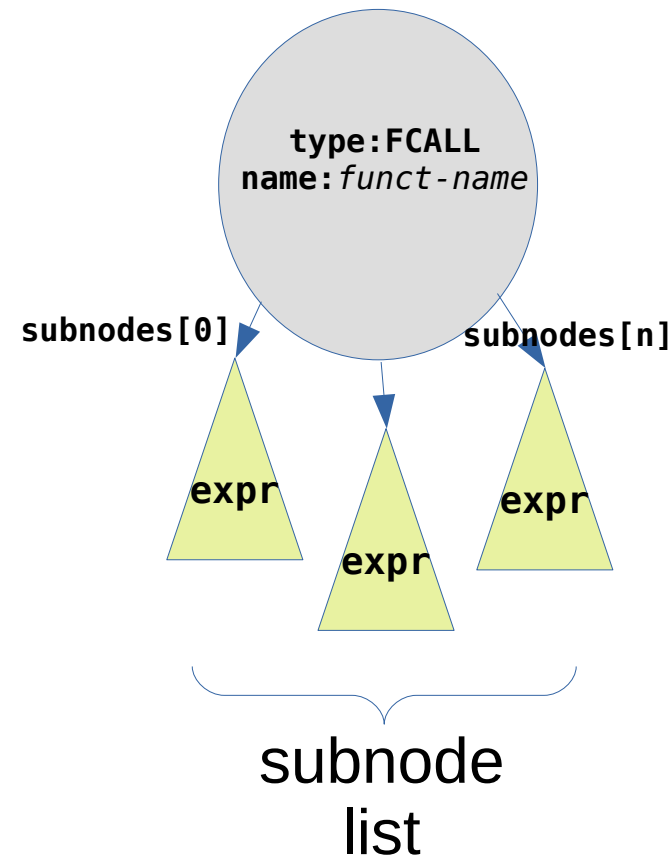
```
function um(x)
    print 0-x
function cube(x) {
    return x*x*x
}
a = 2 + 7.55*44
um(a)
if a-7!=3 or a<355 {
    b = 3*(a-99.01)
    it = 5
    while it>0 {
        print cube(it+b*0.23)
        it = it - 1
    }
}
else
    print a-3.14
```

Κόμβος AST για το return



- Ο τύπος του κόμβου είναι RETURN
- Υπάρχει μοναδικό παιδί (subnodes[0]) με ένα υποδέντρο αριθμητικής έκφρασης (Lexpr)
 - Υπολογίζει την επιστρεφόμενη τιμή

Κόμβος AST για την κλήση συνάρτησης



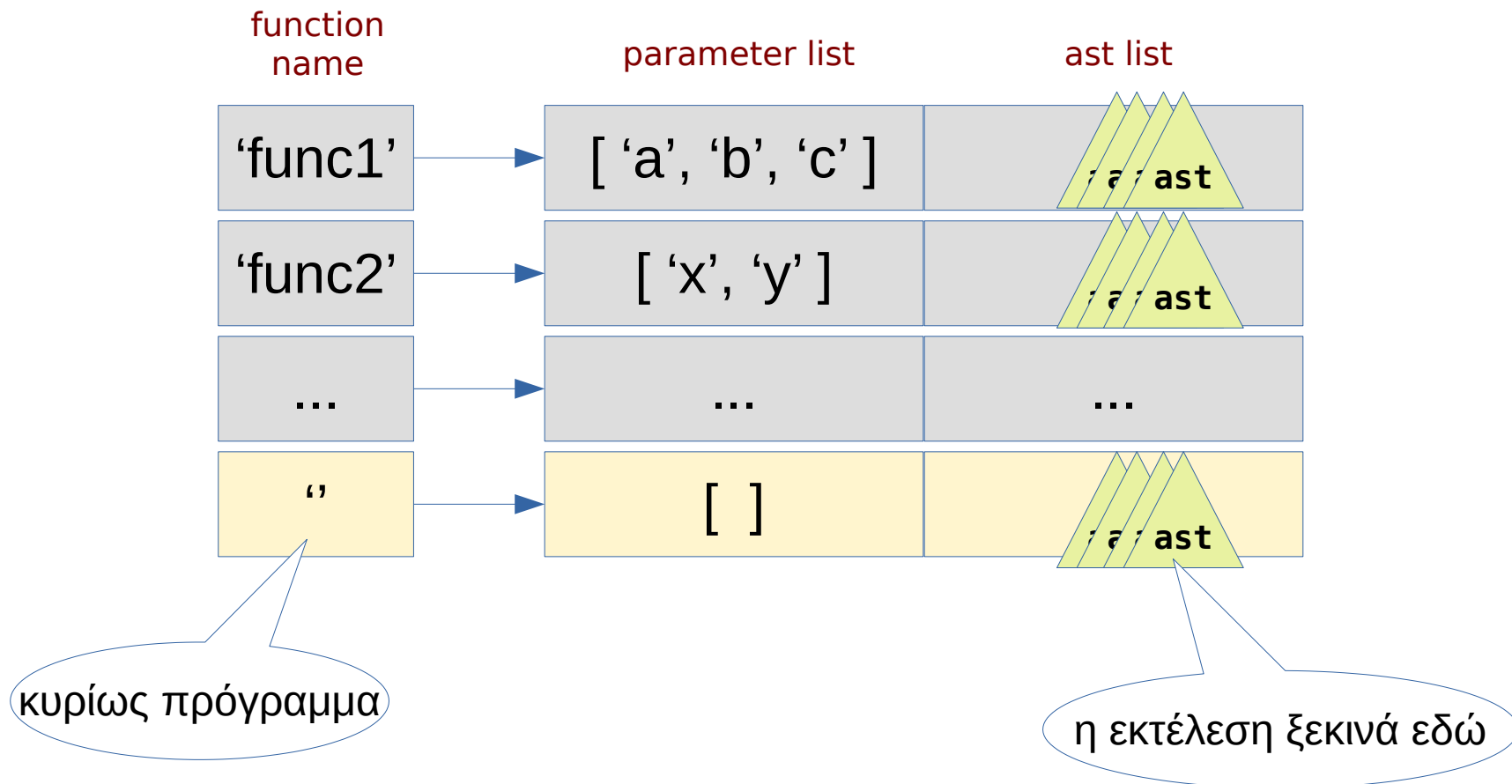
- Ο τύπος του κόμβου είναι FCALL
- Το attribute “name” περιέχει το όνομα της καλούμενης συνάρτησης
- Τα παιδιά (subnodes) του κόμβου είναι μια σειρά από υποδέντρα αριθμητικής έκφρασης, ένα για κάθε όρισμα κλήσης
 - με τη σειρά που αντιστοιχούν στις δηλωμένες παραμέτρους της συνάρτησης

Πληροφορία κάθε συνάρτησης

- Τι παράγει τώρα ο συντακτικός αναλυτής;
 - Προηγουμένως επιστρεφόταν μια λίστα με AST εντολών του κυρίως προγράμματος
- Ποια πληροφορία παράγεται τώρα;
 - Η συντακτική ανάλυση της **δήλωσης** μιας συνάρτησης δημιουργεί μια νέα **λίστα με AST εντολών** για κάθε συνάρτηση
 - Η πληροφορία κάθε συνάρτησης περιέχει επίσης μια λίστα με **τα ονόματα των παραμέτρων** της
- Οι δύο αυτές πληροφορίες αποθηκεύονται σε ένα python dict (**πίνακα συναρτήσεων**) για **κάθε συνάρτηση** που δηλώνεται
 - Ο συντακτικός αναλυτής θα πρέπει να προειδοποιεί μέσω σφάλματος για **πολλαπλές δηλώσεις** της ίδιας συνάρτησης

Πίνακας συναρτήσεων

- Ένα python dict με την πληροφορία κάθε συνάρτησης
 - Το κυρίως πρόγραμμα αποθηκεύεται ως συνάρτηση " (κενό string)



Έλεγχος ορθής κλήσης συναρτήσεων

- Στη γλώσσα μας δεν υποστηρίζονται “**forward declarations**”
 - Κατά τη συντακτική ανάλυση μιας **κλήσης συνάρτησης** είναι πιθανό η καλούμενη συνάρτηση να μην έχει δηλωθεί ακόμα
 - Ο **έλεγχος ύπαρξης** της καλούμενης συνάρτησης και του σωστού αριθμού ορισμάτων πρέπει να μετατεθεί σε **μεταγενέστερο στάδιο**
 - Με τον ίδιο τρόπο, η **ύπαρξη ή όχι επιστρεφόμενης τιμής** μετατίθεται επίσης σε μεταγενέστερη φάση
 - Αν δηλαδή χρησιμοποιήθηκε εντολή **return** πριν το τέλος της συνάρτησης

AST Interpreter

- Προσθήκη λειτουργικότητας για την **κλήση συναρτήσεων** (κόμβοι FCALL)
 - Απαιτείται επιστρεφόμενη τιμή όταν είναι μέρος έκφρασης
- Προσθήκη λειτουργικότητας για την **επιστροφή τιμών** από συναρτήσεις (κόμβοι RETURN)
 - Και έλεγχος της **ροής εκτέλεσης**
 - Μετά από return **δεν πρέπει να εκτελεστούν οι επόμενες εντολές**

Εμβέλεια μεταβλητών

- Στη γλώσσα μας για λόγους απλότητας θα υπάρχουν **μόνο τοπικές μεταβλητές**
 - Κάθε συνάρτηση θα έχει τις δικές της μεταβλητές
 - Για την ακρίβεια: κάθε κλήση μιας συνάρτησης θα έχει τις δικές της μεταβλητές
 - Η μόνη επικοινωνία μεταξύ συναρτήσεων και του κυρίως προγράμματος θα γίνεται **μέσω παραμέτρων και επιστρεφόμενων τιμών**
 - **Δεν** θα υποστηρίζονται global μεταβλητές

Χώρος αποθήκευσης τοπικών μεταβλητών

- Όλες οι «δομημένες» γλώσσες προγραμματισμού χρησιμοποιούν μια στοίβα στη μνήμη (**runtime stack**) για την αποθήκευση των τοπικών μεταβλητών τους
- Στη στοίβα δεσμεύεται χώρος για τις **τοπικές μεταβλητές** σε κάθε κλήση συνάρτησης (**call frame** ή activation record)
- Στη επιστροφή από μια κλήση συνάρτησης το τρέχον call frame **αποδεσμεύεται**
 - Η στοίβα επιστρέφει στην κατάσταση **πριν** την τελευταία κλήση
- Στα call frames αποθηκεύονται επίσης οι **παράμετροι εισόδου** στη συνάρτηση και η **επιστρεφόμενη τιμή**
 - Σημείωση: οι σύγχρονοι μεταγλωττιστές προσπαθούν μέσω βελτιστοποιήσεων να κρατούν μεγάλο μέρος από τα παραπάνω σε **καταχωρητές** αντί της στοίβας (κύρια μνήμη)

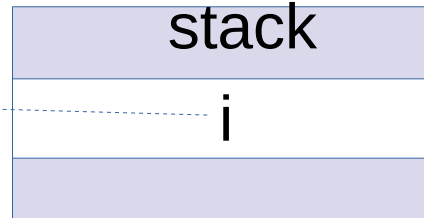
Παράδειγμα

```
f() {  
  int i;  
  ...  
  g();  
}
```

```
g() {  
  int i;  
  ...  
  h();  
}
```

```
h() {  
  int i;  
  ...  
  h();  
}
```

runtime



τρέχον call
frame
κλήσης f()

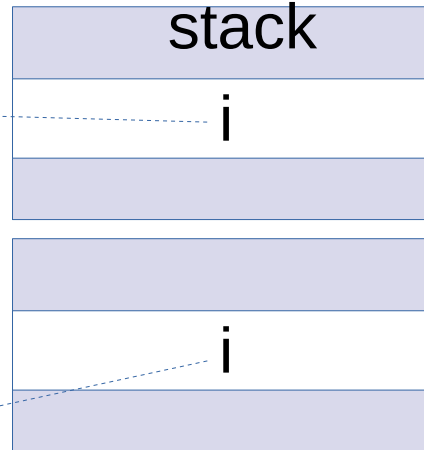
Παράδειγμα

```
f() {  
  int i;  
  ...  
  g();  
}
```

```
g() {  
  int i;  
  ...  
  h();  
}
```

```
h() {  
  int i;  
  ...  
  h();  
}
```

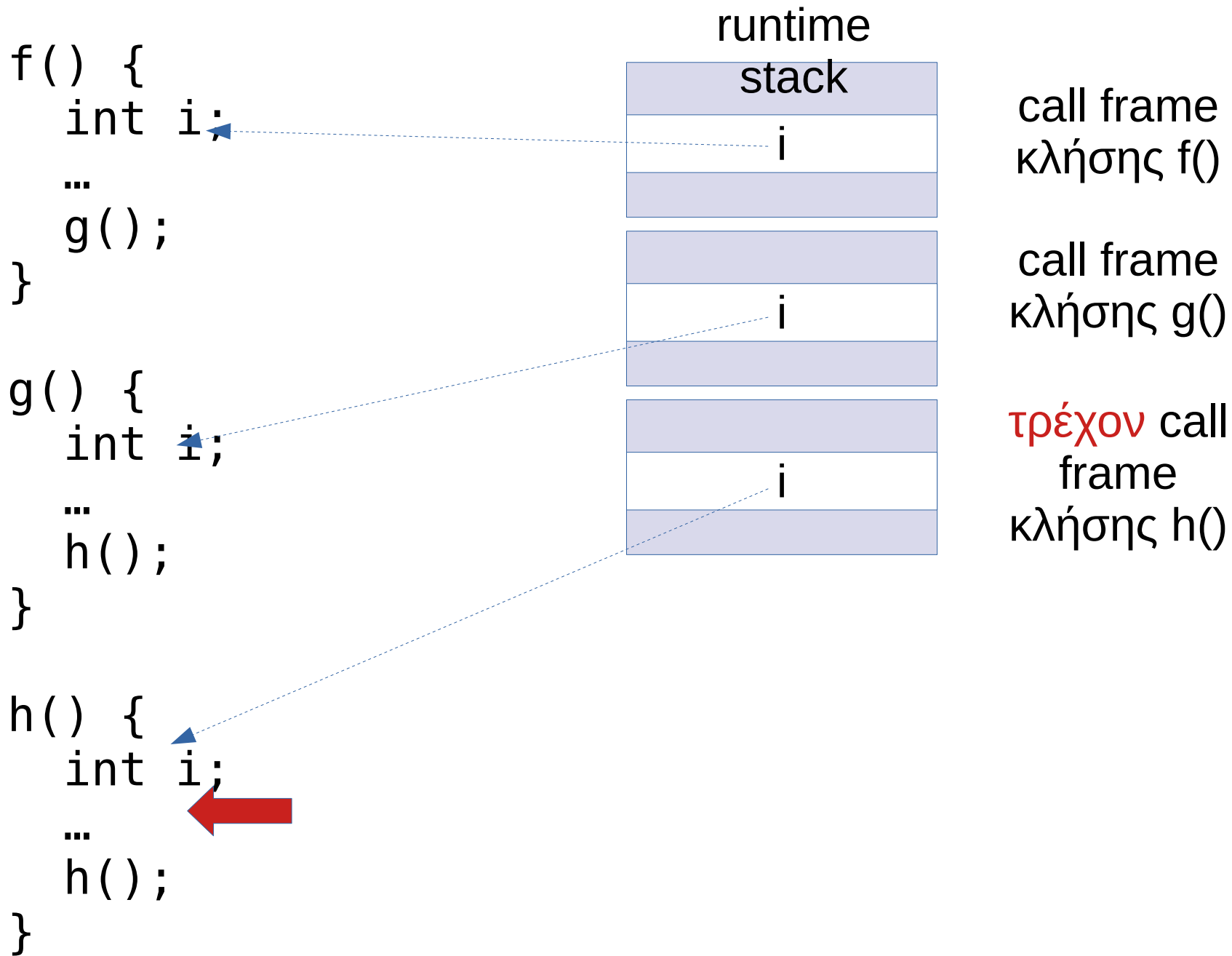
runtime



call frame
κλήσης f()

τρέχον call
frame
κλήσης g()

Παράδειγμα



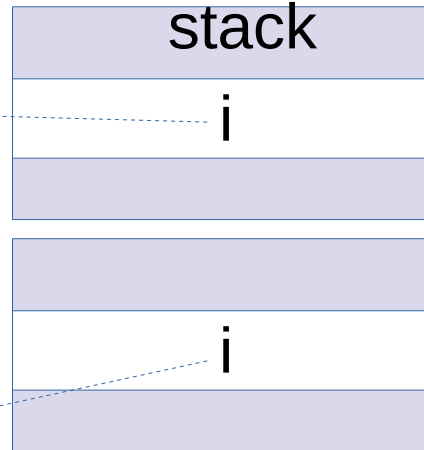
Παράδειγμα

```
f() {  
  int i;  
  ...  
  g();  
}
```

```
g() {  
  int i;  
  ...  
  h();  
}
```

```
h() {  
  int i;  
  ...  
  h();  
}
```

runtime

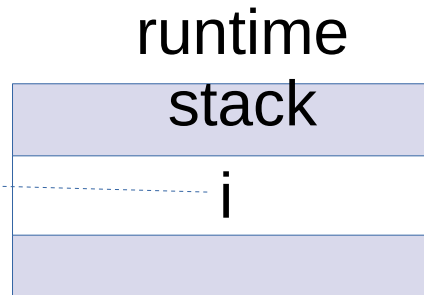


call frame
κλήσης f()

τρέχον call
frame
κλήσης g()

Παράδειγμα

```
f() {  
  int i;  
  ...  
  g();  
}
```



τρέχον call
frame
κλήσης f()

```
g() {  
  int i;  
  ...  
  h();  
}
```

```
h() {  
  int i;  
  ...  
  h();  
}
```

Κλάση RuntimeStack

- Θα τη βρείτε στις βιβλιοθήκες του εργαστηρίου
- Παρέχει ένα απλό API για τη διαχείριση ξεχωριστών χώρων συμβόλων (τοπικών μεταβλητών και των τιμών τους) ανά κλήση συνάρτησης

```
from compilerlabs import RuntimeStack
```

RuntimeStack API

- Δημιουργία νέου αντικειμένου RuntimeStack
`rs = RuntimeStack()`
 - Το νέο runtime stack θα αποτελείται από ένα μοναδικό call frame (αντιστοιχεί στο κυρίως πρόγραμμα)

RuntimeStack API (2)

- Προσθήκη νέου call frame

`rs.push_frame(zip(param_list, arg_values))`

- Το νέο call frame τοποθετείται στην κορυφή του runtime stack (γίνεται το τρέχον frame)
- Το προαιρετικό όρισμα του `push_frame()` αρχικοποιεί το νέο frame με τα ονόματα μεταβλητών του `param_list` και τις αντίστοιχες τιμές τους (`arg_values`)
 - Είναι λίστα από ζευγάρια (όνομα μεταβλητής, τιμή μεταβλητής)
 - Χρησιμοποιείται για να αρχικοποιήσουμε τις παραμέτρους των συναρτήσεων

RuntimeStack API (3)

- Διαγραφή τρέχοντος call frame

`rs.pop_frame()`

- Επιστροφή τιμής μεταβλητής στο τρέχον call frame

`value = rs.dereference(varname)`

– Εάν δεν υπάρχει το `varname` επιστρέφεται `None`

- Ανάθεση τιμής μεταβλητής στο τρέχον call frame

`rs.assign(varname, value)`

RuntimeStack API (4)

- Ανάθεση return value στο τρέχον call frame
`rs.return_value = value`
- Ανάκτηση return value από το τρέχον call frame
`rv = rs.return_value`
 - Εάν δεν έχει τεθεί ως τώρα, επιστρέφεται None
 - Προσοχή: η ανάκτηση του return_value πρέπει να γίνει **πρίν τη διαγραφή του τρέχοντος call frame**
 - δηλ. **πριν την κλήση** του pop_frame()

Διαδικασία κλήσης συνάρτησης

- **Έλεγχος**: υπάρχει το όνομα της συνάρτησης στον πίνακα συναρτήσεων;
- **Έλεγχος**: ο αριθμός των παραμέτρων στη δήλωση της συνάρτησης είναι ίσος με τον αριθμό των ορισμάτων κλήσης;
- **Υπολογισμός** κάθε ορίσματος κλήσης (subnodes στο FCALL)
- **Προσθήκη** (push) ενός νέου call frame, αρχικοποιημένου με τα ονόματα των παραμέτρων και τις αντίστοιχες τιμές των ορισμάτων κλήσης
- **Εκτέλεση** των AST της καλούμενης συνάρτησης
- **Αποθήκευση** της επιστρεφόμενης τιμής (return value)
- **Διαγραφή** (pop) του call frame
- **Επιστροφή** του return value

ControlFlow enumeration

- Χρειαζόμαστε έναν μηχανισμό για να διακόπτουμε την εκτέλεση μετά από **return**
 - Σε οποιοδήποτε βάθος (nesting level)
- Μέχρι τώρα η εκτέλεση των εντολών (συνάρτηση `execute_statements()`) δεν επέστρεφε κάποια τιμή
- Τώρα θα χρησιμοποιήσουμε την επιστρεφόμενη τιμή για να ελέγξουμε τη ροή εκτέλεσης μετά το `return`

```
from enum import Enum
```

```
ControlFlow = Enum('ControlFlow', 'NORMAL RETURN')
```

- Το προηγούμενο δημιουργεί τις τιμές enumeration:
 - `ControlFlow.NORMAL` → η εκτέλεση συνεχίζεται κανονικά
 - `ControlFlow.RETURN` → η εκτέλεση διακόπτεται, επιστροφή προς τα πίσω

Έλεγχος ροής ανά τύπο εντολής

ASSIGN PRINT FCALL	Χωρίς διακοπή στη ροή εκτέλεσης, δεν γίνεται εκτέλεση <i>sub-block</i> κώδικα (στην περίπτωση της <i>FCALL</i> , τυχόν <i>return</i> μέσα στη συνάρτηση δεν αφορά τον κώδικα που την κάλεσε).
IF	Αν γίνει εκτέλεση του <i>sub-block</i> κώδικα του <i>if</i> και επιστρέψει <i>ControlFlow.RETURN</i> , τότε επιστρέφουμε το ίδιο. Αλλιώς συνεχίζουμε χωρίς διακοπή εκτέλεσης.
IFELSE	Αν το <i>sub-block</i> που εκτελέστηκε (είτε με το <i>if</i> είτε με το <i>else</i>) επιστρέψει <i>ControlFlow.RETURN</i> , τότε επιστρέφουμε το ίδιο. Αλλιώς, συνεχίζουμε χωρίς διακοπή εκτέλεσης.
WHILE	Σε κάθε επανάληψη του <i>while</i> ελέγχουμε αν το <i>sub-block</i> που εκτελέστηκε επέστρεψε <i>ControlFlow.RETURN</i> . Αν ναι, διακόπτουμε το <i>while</i> και επιστρέφουμε το ίδιο. Αν δεν συμβεί το παραπάνω, συνεχίζουμε χωρίς διακοπή εκτέλεσης.
RETURN	Επιστρέφουμε <i>ControlFlow.RETURN</i> .